

CLASSIFICATION: ZERO-KNOWLEDGE ARCHITECTURE

P R O T O C O L W H I T E P A P E R

SEES

SECURE END-TO-END STATE

A Zero-Knowledge Vault Architecture for Private Files & Notes

0 PERSONAL DATA REQ.	256-bit LOCAL ENCRYPTION	INF. DEVICES, ONE VAULT
--------------------------------	------------------------------------	-----------------------------------

FIELD VERIFICATION

Sees

Authenticated Protocol Document — sees.im

REV. 0.1 • FILE SE-0.1-2026

STATUS: ACTIVE

EXECUTIVE SUMMARY

SEES (Secure End-to-End State) is a zero-knowledge encrypted vault for files and notes. It is built on a single premise: a service provider should not be able to read user data even if compelled to, compromised, or curious. There is no email address, no phone number, no recovery flow, and no administrative backdoor. The user's passphrase is transformed into an encryption key inside the browser and never leaves the device; only ciphertext is ever transmitted or stored.

This dossier documents the problem SEES addresses, its architecture, its cryptographic protocol, its threat model, its data-handling posture, and open operational questions – filed here for the record.

Every claim in this document is written to be checked rather than taken on faith: the protocol is described in enough detail that its guarantees, and its limits, can be verified independently once an audit is published.

[01] THE PROBLEM

Most “secure” cloud storage and note-taking services are secure only against outside attackers. The provider itself typically retains the ability to read user content – through server-side encryption keys it controls, plaintext database backups, or account-recovery mechanisms that require a copy of the key to be held in escrow. This creates a structural weak point: a subpoena, an insider, a misconfigured database, or a compromised admin panel can expose user data even when transport encryption and access controls are functioning exactly as designed.

A genuinely private vault has to remove this weak point at the architecture level rather than through policy. If the operator never possesses a readable copy of the key or the data, there is nothing to compel, leak, or misuse on the server side.

[02] ARCHITECTURE OVERVIEW

SEES organizes its protocol into four stages, each of which narrows what the server is able to see or do.

01 Vault ID	02 Passphrase	03 PBKDF2	04 AES Key	05 Ciphertext
-----------------------	-------------------------	---------------------	----------------------	-------------------------

Fig. 1 – The five-step path from identity to stored ciphertext. No plaintext crosses the boundary after step 4.

2.1 IDENTITY – CLAIM A NAME, REVEAL NOTHING

A Vault ID functions purely as a locator, not a profile. No email, phone number, recovery address, or personal account is ever attached to it, so the identity layer carries no personally identifying metadata for the server to store or disclose.

2.2 SEAL – ENCRYPT HERE, THEN TRANSMIT

The user's passphrase is stretched locally, inside the browser, into a 256-bit AES key. That key remains in memory only for the duration of the unlocked session. Every byte of user content is sealed before it ever touches the network, so the plaintext never exists outside the user's device.

2.3 DISTRIBUTE – STORE EVERYWHERE, TRUST NOWHERE

Resulting ciphertext is distributed across Storj, a decentralized storage network. Individual storage nodes hold only fragments of ciphertext that they cannot independently decrypt or reassemble into meaningful content, and SEES's own infrastructure never receives the key needed to do so either.

2.4 SHARE – ONE LINK, NO SERVER SECRET

Any note can be turned into a self-contained share link without creating an account or exposing the vault. A fresh encryption key is generated for that link and travels only inside the URL fragment – the part of an address a browser never sends to a server. Shares can optionally be locked with their own password, derived the same way as a vault passphrase, given an expiry, or revoked at any time by their creator. SEES's servers store only a random share identifier and a hash proving revocation authority – never the note content, the key, or the password.

[03] CRYPTOGRAPHIC PROTOCOL

The key-derivation and encryption pipeline favors well-vetted primitives over novel cryptography:

- Key derivation – PBKDF2-SHA256 with 250,000 iterations and a per-vault salt, executed entirely client-side.
- Encryption – AES-256-GCM, an authenticated cipher providing both confidentiality and tamper detection: if ciphertext is altered in transit or at rest, decryption fails rather than silently returning corrupted data.
- Key lifetime – the derived AES key exists only in browser memory while the vault is unlocked, and is never persisted or transmitted.

Summarized: passphrase + salt → PBKDF2-SHA256 (250,000 rounds) → AES-256 key (memory-only) → AES-256-GCM ciphertext → distributed storage.

TAMPER-EVIDENT SEAL — AES-GCM AUTHENTICATION VERIFIED

[04] THREAT MODEL

SEES is explicit about what its architecture defends against and what remains the user's responsibility — a distinction that is itself part of the security design.

DESIGNED TO RESIST

- A hostile or compelled server operator, since the server never holds a usable key.
- Inspection of individual storage nodes, since each node holds only undecryptable ciphertext fragments.
- Network interception, since only ciphertext ever crosses the wire.
- Ciphertext tampering, caught by AES-GCM's authentication tag.

REMAINS USER-CONTROLLED

- Device malware capable of reading memory or keystrokes locally.
- Weak or reused passphrases, which reduce the derived key's effective strength.
- Malicious or compromised browser extensions with page access.
- Voluntary sharing of credentials by the user.

Because there is no recovery key, no plaintext backup, and no administrative override, a lost passphrase means the vault's contents are unrecoverable by design – the same property that keeps the provider from reading it also keeps the provider from rescuing it.

[05] DATA HANDLING & PRIVACY

SEES does not request an account identity. No email, phone number, or profile is required to operate a vault. Ordinary connection metadata – such as IP address and user agent – is still visible to the service, as it is to any web application that must respond to a request.

5.1 WHAT REMOTE STORAGE ACTUALLY HOLDS

- Encrypted blobs – the sealed notes and activity data themselves, unreadable without the local key.
- Opaque identifiers – used to locate vault records without exposing plaintext content.
- KDF metadata – salt and iteration count, which are required for key derivation but are not secret.

5.2 THIRD PARTIES

Storj stores ciphertext on behalf of the vault; hosting infrastructure serves the application itself. Both may process operational metadata needed for delivery and security, but neither receives the passphrase or the decryption key at any point.

NO ADS – NO PROFILES – NO BACKDOORS – NO PLAINTEXT TROVE

[06] OPERATIONAL FAQ

Q1. What happens if I forget my passphrase?

The vault becomes unreadable. There is no reset email and no recovery key – this is a direct consequence of the zero-knowledge design, not an oversight.

Q2. Can SEES read my notes?

No. Notes are encrypted locally before storage, and SEES does not receive the key at any stage.

Q3. Can I use multiple devices?

Yes. The same Vault ID and passphrase let the browser derive the same key locally on any device.

Q4. Does SEES support two-factor authentication?

Yes, via an authenticator app from Vault Settings. It adds a second local unlock step but is not account recovery and cannot replace the passphrase.

Q5. Does SEES work offline?

Previously cached encrypted notes can be read after unlock. New writes currently require a connection.

Q6. Has SEES been independently audited?

Not yet. It should be treated as early-stage software until an independent audit is published.

Q7. Will email login be added?

No. Email-based recovery would require an account and a recovery path, which works against the zero-knowledge model by design.

Q8. Can I share a note with someone else?

Yes. A share link embeds its own decryption key in the URL fragment, which never reaches SEES's servers. Links can be password-protected, given an expiry, and revoked by their creator at any time.

[07] GLOSSARY OF TERMS

Zero-Knowledge	An architecture in which the service provider never possesses the information needed to read user data, even in principle.
PBKDF2-SHA256	A key-derivation function that repeatedly hashes a passphrase with a salt to slow down brute-force attacks.
AES-256-GCM	An authenticated symmetric cipher providing both confidentiality and tamper detection in a single pass.
Vault ID	A bare locator used to find a vault's data; not tied to any personal

	identity or account.
Storj	A decentralized storage network that holds encrypted data fragments across independent nodes.
Share Link	A URL whose fragment carries a note's encryption key (or, if password-protected, only a salt); browsers never transmit the fragment to a server, so the secret stays with whoever holds the link.

[08] COMPARATIVE POSITION

Filed for reference: how the SEES model compares against the typical architecture of a conventional cloud storage or notes service on the dimensions that matter most for confidentiality.

Dimension	SEES	Conventional Cloud
Key custody	Held only by user, in-browser	Held or escrowed by provider
Server-side plaintext	Never exists	Often exists for search/preview
Account identity	None – bare Vault ID	Email, phone, or SSO profile
Password reset	Not possible by design	Provider-mediated reset flow
Breach exposure	Ciphertext fragments only	Potentially readable records
Subpoena response	No readable data to hand over	Provider can often comply

Fig. 2 – Positional comparison, not a claim about any specific competing product.

[09] RISK REGISTER

A dossier is only honest if it files the risks alongside the guarantees. The register below restates the threat model as discrete entries, each with its residual likelihood and its mitigation of record.

Risk	Residual	Mitigation of Record
Lost passphrase	High (user-side)	None by design – no recovery path exists
Weak passphrase choice	Medium	250,000-round PBKDF2 slows brute force
Device compromise	Medium	Outside architecture scope – user responsibility
Server compromise	Low	No usable key or plaintext ever resides server-side
Storage-node inspection	Low	Nodes hold undecryptable ciphertext fragments only
Ciphertext tampering	Low	AES-GCM authentication tag causes decryption failure
Share link exposure	Medium (user-side)	Anyone holding the link can decrypt it – mitigated by optional password, expiry, and owner revocation

Fig. 3 – Risk register, current as of this document's revision.

[10] DOCUMENT CONTROL

This section records the dossier's own provenance – the paperwork behind the paperwork.

Field	Value	Notes
Document	SEES White Paper	Protocol overview, v0.1
Classification	Zero-Knowledge	No confidential contents – public spec
Audit status	Unaudited	Independent audit not yet published
Last updated	29 Jul 2026	Adds the zero-knowledge sharing protocol (section 2.4)
Distribution	Unrestricted	Suitable for public / partner review

Fig. 4 – Document control record. Retained for internal reference and version tracking.

FILED & SEALED – SE-0.1 – 2026

[11] CONCLUSION

SEES applies a straightforward principle with unusual consistency: if the provider never holds the key or the plaintext, the provider cannot be the point of failure for either. Identity is reduced to a bare locator, encryption happens before data leaves the device, and storage is distributed across nodes that individually see nothing usable. The result is a vault whose confidentiality does not rest on trust in the operator – only on the strength of the user's own passphrase and the security of their device.

The logo for 'Sees' is written in a large, elegant, black cursive script. A thin, horizontal line with a slight upward curve at the right end is positioned directly beneath the word.

END OF DOSSIER – sees.im – 2026